

```
Algorithm main()
{
  write "enter array size:"
  read n
  for i := 0 to n-1 do
  {
    write "enter element at index", i, ":"
    read A[i]
  }
  result := missingIntegerRecursive(A, n)
  write "the smallest missing positive integer is", result
}

Algorithm missingIntegerRecursive(A, n)
{
  return findMissing(A, n, 1)
}

Algorithm findMissing(A, n, current) {
  if current > n+1 then
    return current
  if isFound(A, n, current) == 0 then
    return current
  return findMissing(A, n, current + 1)
}

Algorithm isFound(A, n, Val) { if n=0 then return 0
  if A[0] = Val then
    return 1
  return isFound(A+1, n-1, Val)
}
```

Date

No Analysis

* Basic operation is the comparison
if ($A[0] == \text{val}$)

If function is found each recursive call decreases
recurrence is: $T(n) = T(n-1) + c$

$$= T(n-2) + 2c$$

$$= T(n-3) + 3c$$

$$= T(0) + n \cdot c$$

$$= O(n)$$

If finding for every value of current is found called
and takes $O(n)$ because is found call and recursive call
recurrence relation $T(n) = T(n-1) + n$

$$= T(n-2) + T(n-1) + n$$

$$= T(n-3) + T(n-2) + T(n-1) + n$$

$$T(n) = \frac{n(n+1)}{2} = O(n^2)$$

$$\text{Time complexity} = O(n^2)$$

↳ for overall algorithm

* If 1 is missing Positive integer: best case = $O(n)$

* if array contains all nums from 1 to n?

$$\text{worst case} = O(n^2)$$